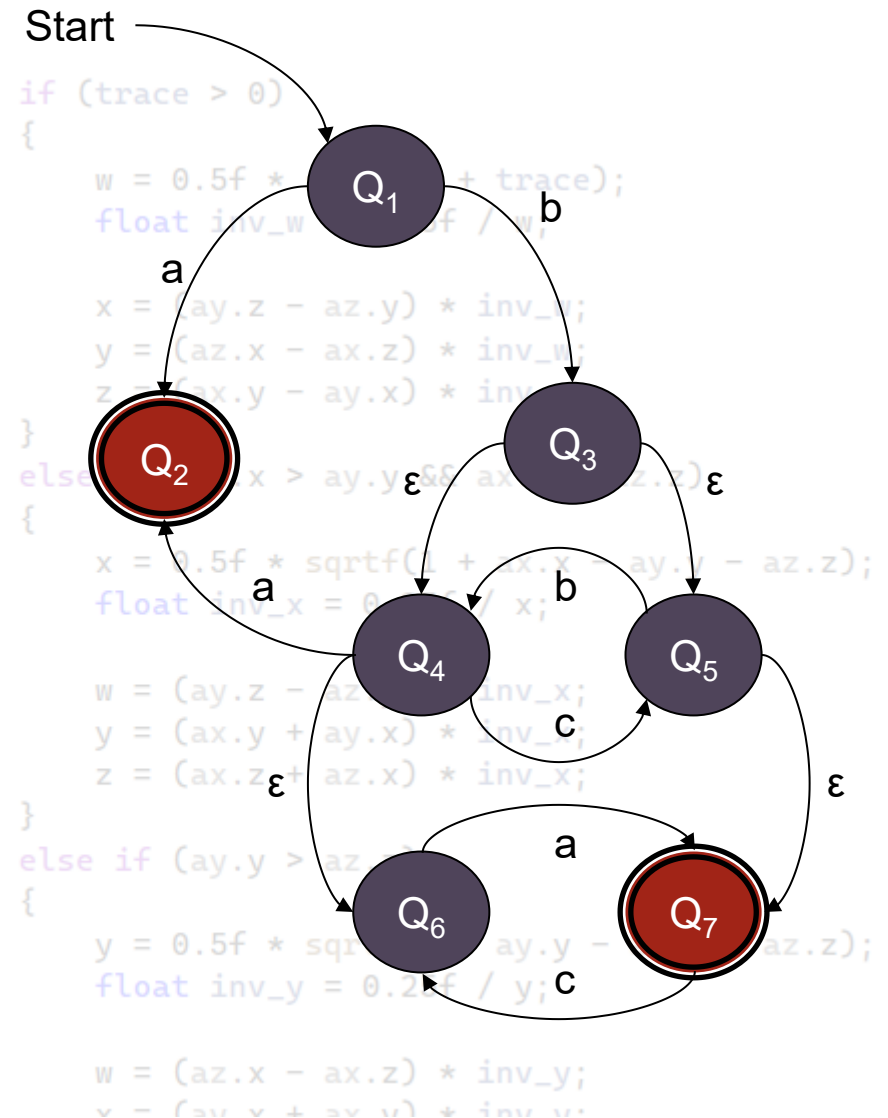


Algorithmen und Datenstrukturen

4 – Arrays

Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de



Einleitung

Heranführung

- Sei `a` ein Array mit 10 Einträgen. Was haben `a[5 + 2]` und `a[2 * 8 - 9]` gemein?
- Welche Probleme hat die folgende Funktion?

```
// Get starting point and ending point for robot movement.  
void get_robot_path(waymark* start, waymark* end)
```
- Was ist eine terminierende Null?
- Was könnte man statt einer terminierenden Null verwenden?

Kompetenzen

- Referenzen und konstante Zugriffswege benutzen
- dynamische Arrays und Arrays fester Größe in C++ korrekt verwenden
- interne Umsetzung von Strings verstehen
- mehrdimensionale Daten im Speicher ablegen

Vortragsübersicht

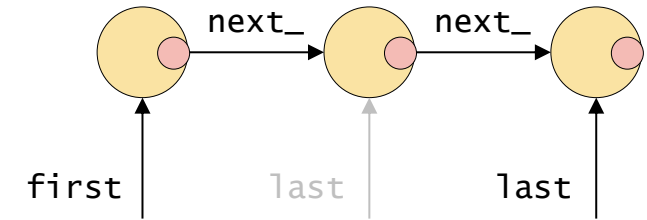
4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Referenzen und Konstantheit (1)

Mehrfache Namen für bestehende Variablen

```
struct waymark { float x, y; waymark* next; }  
int main()  
{  
    waymark *first = create_waymark(), *last = first;  
    last->next = create_waymark();  
    last = last->next;  
    std::cout << last->x;      // identisches Ziel zu first->next->x  
    last->next = create_waymark();  
    last = last->next;  
    std::cout << last->x;      // identisches Ziel zu first->next->next->x  
}
```



Beobachtungen:

- für die selbe Variable kann es in den meisten Programmiersprachen mehrere gleichwertige *Zugriffswege* geben
- jeder Zugriffsweg gibt der Variable ein *Alias* (d. h. einen Namen) und lässt sich durch jeden gleichwertigen Zugriffsweg ersetzen
- Zuordnung zwischen Zugriffswegen und Variablen ist komplex (z. B. je nach Laufzeit-Wert oder Sichtbarkeit von Variablen)

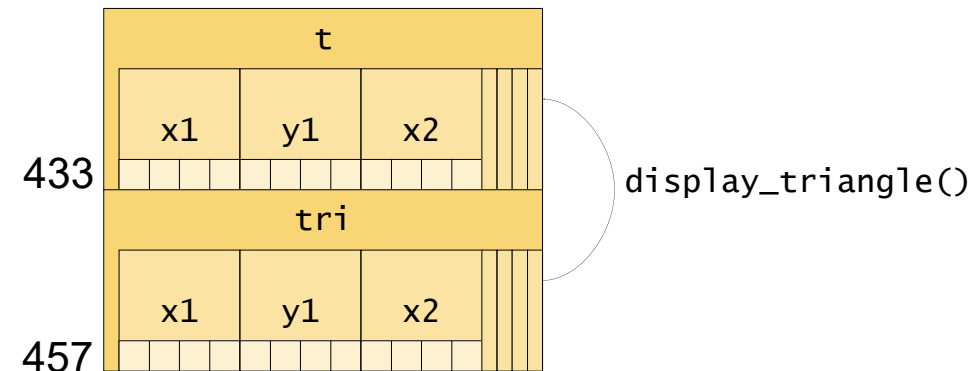
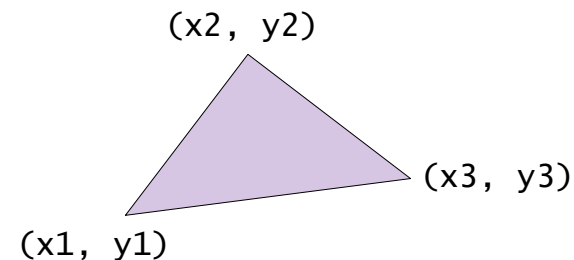
Referenzen und Konstantheit (2)

Teure, unnötige Übergabe als Kopie

```
struct triangle
{ float x1, y1, x2, y2, x3, y3; };
```

```
void display_triangle(triangle t)
{
    line(t.x1, t.y1, t.x2, t.y2);
    line(t.x2, t.y2, t.x3, t.y3);
    line(t.x3, t.y3, t.x1, t.y1);
}
```

```
void pass_by_value()
{
    triangle tri(1, 2, 6, 5, 3, 9);
    display_triangle(tri);
}
```



- Übergabe von `t` als Wert erzeugt funktionslokale Kopie auf dem Stack
- **Vorteil:** Änderungen an `t` bleiben auf die Funktion `display_triangle` beschränkt
→ `t` wird in dieser Funktion aber gar nicht geändert
- **Nachteil:** bei extrem vielen Aufrufen durch unnötige Kopien von `t` langsam
→ Compiler können diese Kopien nicht immer herausoptimieren

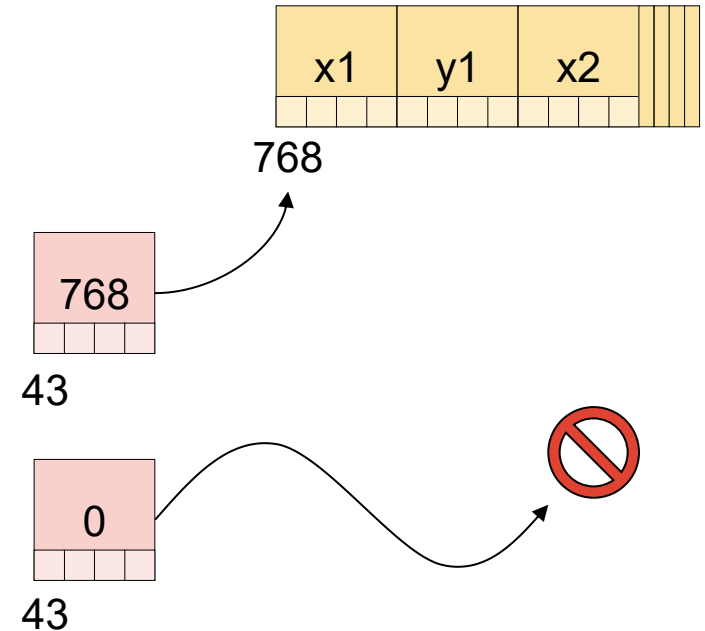
Referenzen und Konstantheit (3)

Schnelle, gefährliche Übergabe als Zeiger

```
struct triangle
{ float x1, y1, x2, y2, x3, y3; };
```

```
// Draw a not-null triangle t.
void display_triangle(triangle *t)
{
    line(t->x1, t->y1, t->x2, t->y2);
    line(t->x2, t->y2, t->x3, t->y3);
    line(t->x3, t->y3, t->x1, t->y1);
}
```

```
void pass_by_pointer()
{
    triangle *tri = nullptr;
    // ...
    display_triangle(tri);    // undef. vh.,
                             // wegen Doku ist pass_by_pointer() schuld daran!
}
```



- Übergabe von `t` als Zeiger erzeugt Zugriffsweg auf bestehende Variable
- **Vorteil:** keine unnötige Kopie mehr
- **Nachteil:** Zugriff auf Ziel von `nullptr` führt zu undefiniertem Verhalten
→ Umgang mit `nullptr` muss geklärt werden (z. B. hier mit Doku)

Referenzen und Konstantheit (4)

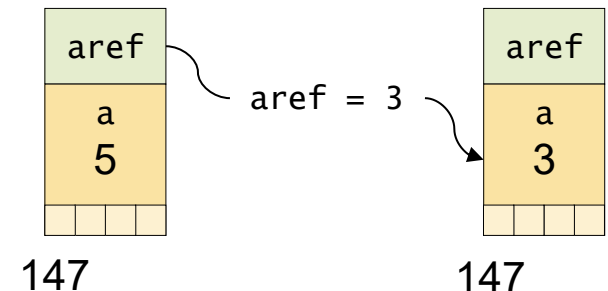
Referenzen als Datentyp

Verbesserungsvorschlag:

- Zeiger geben Zielvariablen über Rechenausdruck (*ptr) einen anderen Namen
- erlaube Programmierern, andere Namen für Variablen direkt zu vergeben
 - neuer Datentyp *Referenz* (direktes Alias für Variable)
 - Achtung: trotz Ähnlichkeiten nicht mit Referenz (Java) verwechseln!

Syntax:

```
int a = 5;
int &error;           // Kompilier-Fehler
int &aref = a;         // OK
aref = 3;             // äquivalent zu a = 3
cout << a << " " << aref; // Ausgabe: 3 3
```



- Referenz deklarieren mit `typesreferenzziels &namederreferenz`
- Referenz **muss** bei Deklaration bzw. Konstruktion initialisiert werden (z. B. = `zielvariable`)
- Referenz ist anschließend immer ein Alias (oder Zugriffsweg) für die `zielvariable`
 - keine Dereferenzierung, Initialisierung ohne Adresse-&, Member-Zugriff mit `.` statt `->`
 - intern manchmal (z. B. bei Membervariablen) trotzdem als Zeiger realisiert

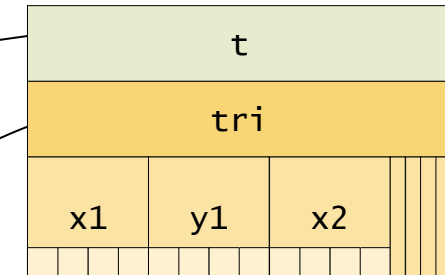
Referenzen und Konstanttheit (5)

Schnelle, sichere Übergabe per Referenz

```
struct triangle
{ float x1, y1, x2, y2, x3, y3; };
```

```
void display_triangle(triangle &t)
{
    line(t.x1, t.y1, t.x2, t.y2);
    line(t.x2, t.y2, t.x3, t.y3);
    line(t.x3, t.y3, t.x1, t.y1);
}
```

```
void pass_by_reference()
{
    triangle tri(1, 3, 7, 4, 8, 8);
    display_triangle(tri);
}
```



768

Vorteile:

- ähnliche Performance wie mit Zeigern (keine Dreiecks-Kopie)
- weniger fehleranfällig, weniger Dokumentationsbedarf, sprechender Code

Faustregel: Bevorzuge Referenzen über Zeiger wann immer möglich.

Referenzen und Konstantheit (6)

Kein Umbiegen von Referenzen

```
class vector  
{ float x, y, z; };
```

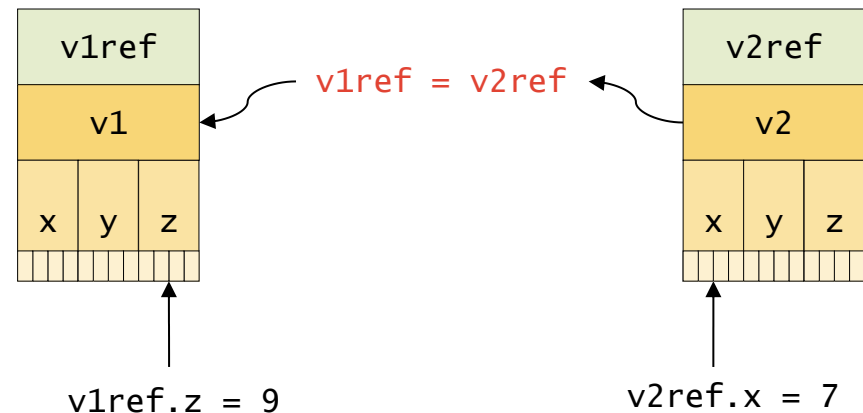
```
vector v1(1, 2, 3);  
vector v2(4, 5, 6);
```

```
vector& v1ref = v1;  
vector& v2ref = v2;
```

```
v1ref = v2ref;    // äquivalent zu v1 = v2  
v1ref = v2;       // äquivalent zu v1 = v2
```

```
v1ref.z = 9;  
v2ref.x = 7;
```

```
// v1 == v1ref == (4, 5, 9)  
// v2 == v2ref == (7, 5, 6)
```



- der Ausdruck `referenzvariable` steht nach der Deklaration bzw. Konstruktion **immer** für die gebundene Variable
 - Referenzen lassen sich nach der Initialisierung nicht mehr umbiegen!
 - stattdessen erfolgt Zuweisung der gebundenen Variablen!

Referenzen und Konstantheit (7)

Konstante Variablen in C++

```
struct answer { std::string content_; /* ... */ };
struct final_test
{
    final_test() : note_(5), answer_("i = (-1) * (-1)") { }
    int note_;
    const answer answer_;
};

final_test t;
t.note_ = 1;
t.answer_.content_ = "i = sqrt(-1)"; // OK, nicht const
// Kompilier-Fehler, const über answer_

const final_test tc;
tc.note_ = 1; // Kompilier-Fehler, const über tc

answer a;
a.content_ = "i = sqrt(-1)"; // OK, in keinem Kontext const
t.answer_ = a; // Kompilier-Fehler, const in final_test
```

Umsetzung in C++:

- Schlüsselwort `const` garantiert, dass eine Variable nicht verändert wird
- bei Klassenmitgliedern bezieht sich ein `const` rekursiv auf alle Member
- Markierung einzelner Instanzen oder einzelner Member möglich

Referenzen und Konstantheit (8)

Konstant markierte Zugriffswege

```
int i = 5, j = 13;
```

```
const int *iptr = &i;
```

```
*iptr = 10; // Kompilier-Fehler: iptr ist const
```

```
const int &iref = i;
```

```
iref = 8; // Kompilier-Fehler: iref ist const
```

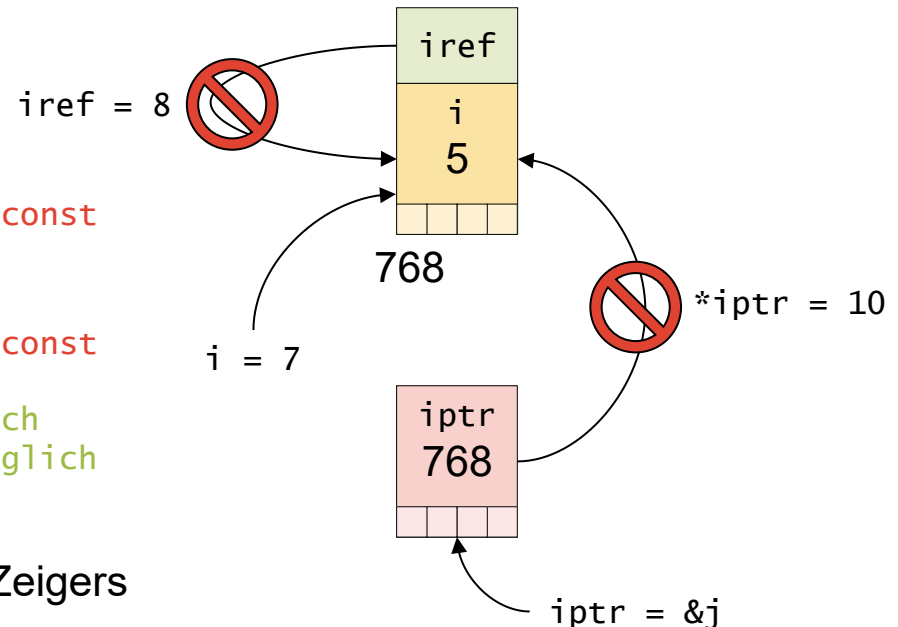
```
i = 7; // Änderung von i bleibt möglich
```

```
iptr = &j; // Änderung von iptr bleibt möglich
```

- Schlüsselwort `const` in dem Zieldatentyp eines Zeigers oder einer Referenz verhindert alle Änderungen am Wert des Ziels über den Zeiger / die Referenz
 - Compilerprüfung sorgt für sicheren Code
 - sprechender Code

Faustregel:

Verwende konstante Zeiger und Referenzen wann immer möglich.



Referenzen und Konstantheit (9)

Übergabe per konstanter Referenz

```
struct triangle
{ float x1, y1, x2, y2, x3, y3; };

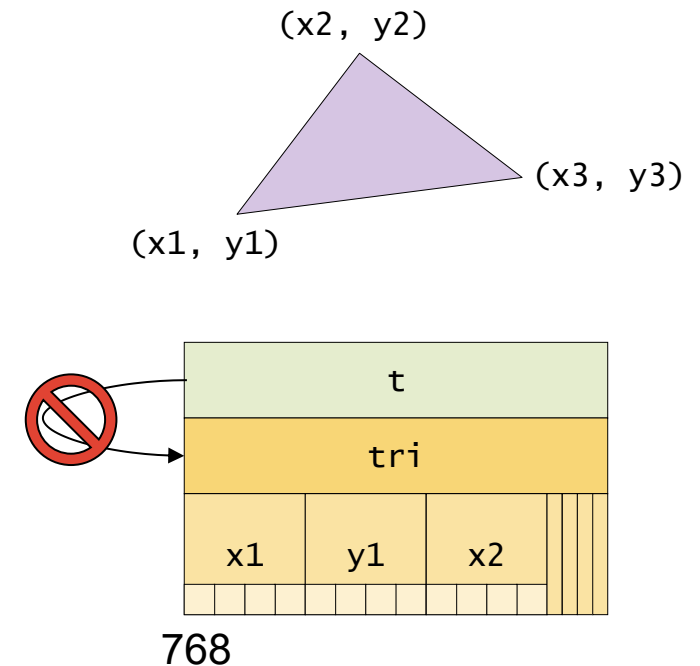
void display_triangle(const triangle &t)
{
    line(t.x1, t.y1, t.x2, t.y2);
    line(t.x2, t.y2, t.x3, t.y3);
    line(t.x3, t.y3, t.x1, t.y1);
}
```

ablesbar aus Übergabetyp:

- keine Kopie von `t` (Performance)
- `t == nullptr` verboten (Korrektheit, Dokumentation)
- keine Änderung am Ziel von `t` (Korrektheit, Dokumentation)

Ergebnis:

Code nun vollständig selbstdokumentierend!



[8]

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Zeigerarithmetik (1)

Arrays im Arbeitsspeicher

gegeben:

- n Werte v_0 bis v_{n-1}
- Werte haben Datentyp τ mit fester Bytegröße s

gesucht:

- Ablage der Werte im Arbeitsspeicher

Rahmenbedingungen:

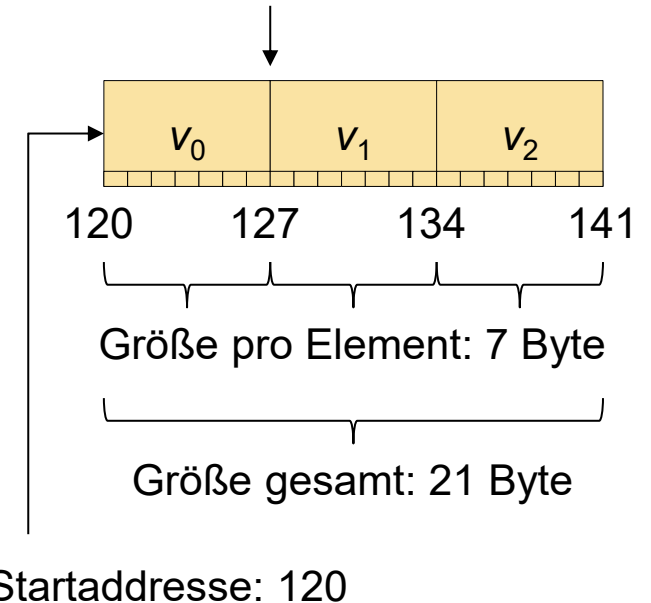
- möglichst kompakte Darstellung im Speicher
- möglichst schneller Zugriff auf einzelne Elemente

Lösung:

- allokiere Speicherbereich mit genügend Speicher für alle Elemente ($n * s$ Byte)
- lege Werte lückenlos in direkt aufeinander folgenden Speicherzellen ab
- finde Speicherbereich über Startadresse des ersten Elements wieder

Beispiel: $n = 3$, $s = 7$

Keine Lücken zwischen Elementen!



Zeigerarithmetik (2)

Elementzugriff im Array

gegeben:

Array nach Darstellung wie zuvor, also:

- Startadresse $\text{adr}(\text{start})$ des ersten Elements
- gleiche Größe s in Bytes für jedes Element
- Elemente liegen geschlossen im Speicher

gesucht:

Adresse $\text{adr}(k)$ von k -tem Element (mit Index $i = k - 1$)

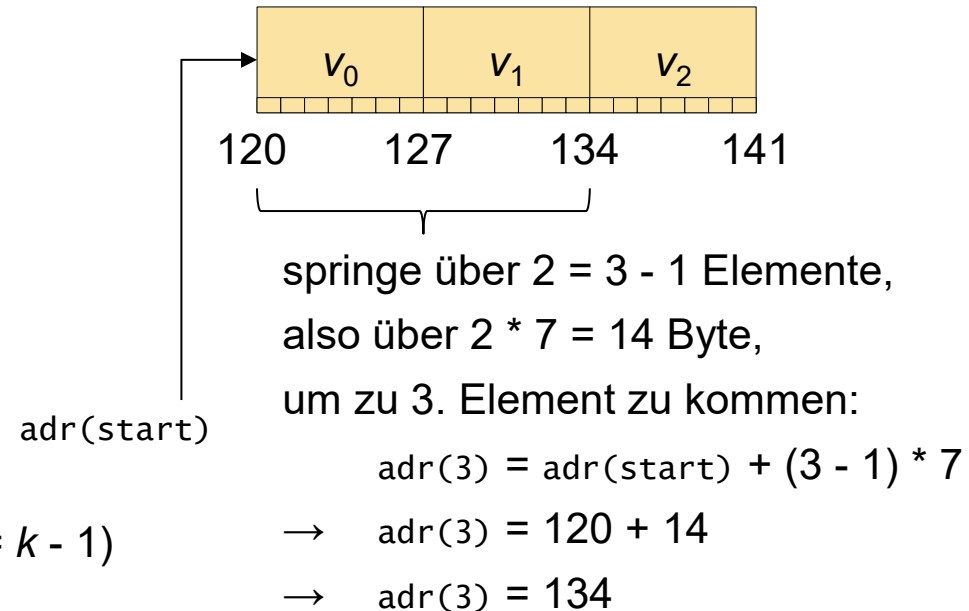
Lösung:

springe über die Bytes von $k - 1$ Elementen, um zu k -tem Element zu kommen:

$$\begin{aligned}\text{adr}(k) &= \text{adr}(\text{start}) + (k - 1) * s \\ &= \text{adr}(\text{start}) + i * s\end{aligned}$$

Achtung: Zwischen Indizes und den Ordnungszahlen unterscheiden!

→ das erste Element hat Index 0, das zweite Element hat Index 1, ...



Zeigerarithmetik (3)

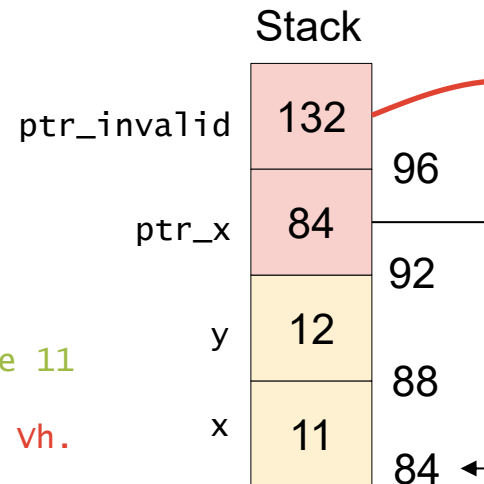
Rechenoperationen mit Zeigern

```
int x = 4;  
int y = 12;
```

```
int *ptr_x = &x;  
int *ptr_invalid = ptr_x + y;
```

```
*ptr_x = 11;           // OK  
std::cout << x;        // Ausgabe 11
```

```
*ptr_invalid = 37;     // undef. vh.
```



$ptr_invalid = ptr_x + y$

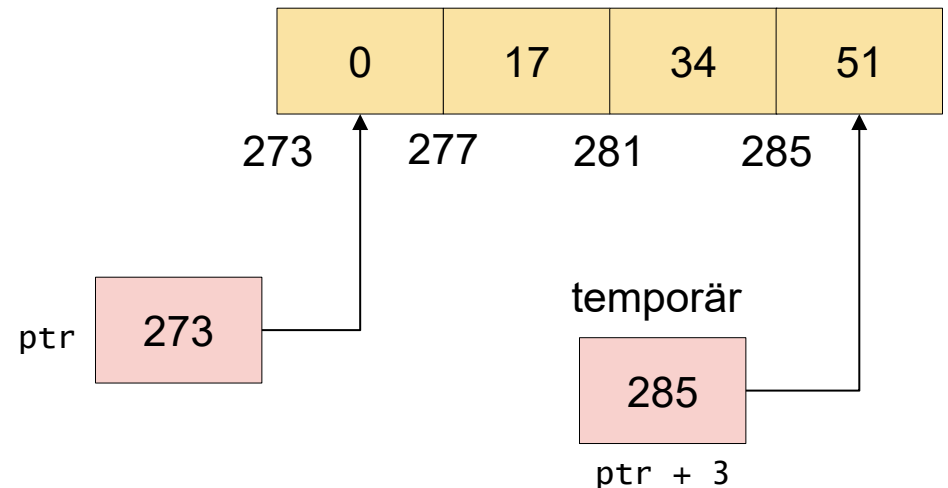
// interne Rechnung:
 $adr(ptr_invalid) =$
 $adr(ptr_x) + sizeof(int) * y =$
 $adr(ptr_x) + sizeof(int) * 12 =$
 $84 + 4 * 12 =$
 132

- **Zeigerarithmetik:** zeiger $\pm$ offset
 - zeiger ist Variable mit beliebigem Zeigertyp (z. B. `int*`) außer `void*`
 - offset ist Variable mit beliebigem Ganzzahltyp (z. B. `short`, `int`)
- Rückgabewert von zeiger $\pm$ offset ist "verschobener" Zeiger
 - Verschiebung beträgt offset * Bytegröße des Zieldatentyps (z. B. 2 Byte bei `short*`)
 - intern: $\pm$ offset * Bytegröße wird zur Adresse in zeiger addiert
- Rückgabezeiger zeigt in der Regel auf nicht allokierten Speicher
 - (nur) bei Zugriff auf Zielvariable: undefiniertes Verhalten

Zeigerarithmetik (4)

Zeigerarithmetik für den Arrayzugriff

```
int *ptr =  
    /* Zeiger auf Array mit 4 Integer */;  
  
// schreibe wert in jeden Integer  
for (int i = 0; i < 4; ++i)  
    *(ptr + i) = (i * 17);  
  
// Zugriff auf vierten Integer,  
// d. h. Integer mit Index 3  
std::cout << *(ptr + 3) << std::endl;
```



gegeben:

- Array wie zuvor (z. B. Zeiger `ptr` auf erstes Element)
- Zeigerarithmetik (also Rechnung `ptr + offset`)

(also: um 3 ints nach rechts
verschobener `ptr`-Zeiger)

Beobachtung:

- Zeigerarithmetik `ptr + offset` berechnet Zeiger auf Element mit Index `offset`
→ überspringe `offset * sizeof(zieltyp)` Bytes
- die Dereferenzierung `*(ptr + offset)` ist Alias für Element mit Index `offset`
→ Zugriff auf Array-Elemente mit Zeigerarithmetik und Dereferenzierung

Zeigerarithmetik (5)

Kurzschreibweise für den Array-Zugriff

```
int *ptr =  
    /* ... Zeiger auf Array mit vier Integern ... */;  
  
for (int i = 0; i < 4; ++i)  
    ptr[i] = (i * 17);  
  
std::cout << ptr[3] << std::endl;  
  
free(ptr);
```

- der Ausdruck `ptr[y]` ist Kurzschreibweise für `*(ptr + y)`
 - deshalb „funktionieren“ Arrays
 - **konzeptuelle Bedeutung:**
Zeigerarithmetik erklärt das Verhalten von Arrays
in C, C++ und den meisten höheren Sprachen!

Faustregel:

Array-Kurzschreibweise wenn möglich verwenden.

Zeigerarithmetik (6)

Kosten der Zeigerarithmetik

interne Umsetzung:

`adresse_ergebnis = adresse_start + bytes_pro_objekt * index`

Theorie:

allgemeine Zahlen addieren und multiplizieren

- Laufzeit für Addieren und Multiplizieren hängt von Stellen (Bits) der Zahlen ab
- Gesamtrechnung hat $O(\text{polynom})$ Laufzeit
- Zugriff auf Array-Element kostet beliebig mehr Laufzeit, je höher Adresse und Index sind

Praxis:

Adresse und Index sind Integer-Variablen mit fester Stellenzahl (z. B. 4 Byte)

- Laufzeit für Addieren und Multiplizieren ist beschränkt
- Gesamtrechnung hat $O(1)$ Laufzeit
- Zugriff auf Array-Element kostet beschränkt viel Laufzeit

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- **Dynamische Arrays**
- Stolpersteine bei dynamischen Arrays
- Arrays fester Größe
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

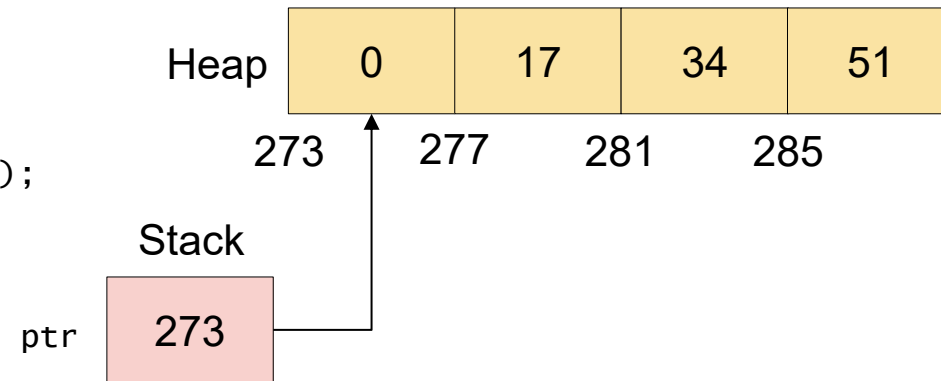
Dynamische Arrays (1)

Allokieren von Array-Speicher

```
int element_count = 4;
int *ptr =
    (int*)malloc(sizeof(int) * element_count);

for (int i = 0; i < 4; ++i)
    ptr[i] = (i * 17);
    // oder: *(ptr + i)

free(ptr);
```



- *dynamisches Array* ist Array mit prinzipiell zur Laufzeit wählbarer Größe (z. B. mit `element_count` aus Benutzereingabe)
- verwende `malloc()`, um Heapspeicher für Array zu allokalieren
 - nutze `sizeof(typ)`, um Größe eines Typs in Byte zu bestimmen
 - allokieren `sizeof(typ) * element_count` Bytes für alle Arrayelemente
 - Rückgabe ist `void*` auf Speicher, Konvertierung in passenden Typ nötig
- Zugriff auf Arrayelemente mit Zeigerarithmetik wie gehabt
- Speicher mit `free()` freigeben, wenn nicht mehr benötigt

Dynamische Arrays (2)

Anzahl der Elemente in einem Array

```
void print_results
(float *results, int result_count) // "results" points to "result_count" elements
{
    for (int i = 0; i < result_count; ++i)
        std::cout << results[i];
}

class point_cloud
{
    point3f *points;
    int number_of_points; // number of elements that "points" points to
};
```

Problem:

es gibt in C++ keine allgemeine Methode, um die Größe eines Arrays zu bestimmen

- mit `sizeof(zeiger)` lässt sich die Arraygröße **nicht** ermitteln (→ später!)
- Systemfunktionen (z. B. `_msize()`) sind nicht standardisiert und limitiert (z. B. nur Heap)

Lösung:

- Zahl der Elemente in separater (Member-)Variablen merken
- bei Übergabe als Zeiger die Zahl der Elemente mitgeben

Dynamische Arrays (3)

Umallokieren dynamischer Arrays

```
int* my_integers =  
    (int*)malloc(sizeof(int) * 7);  
  
// write to all array elements  
for (int i = 0; i < 7; ++i)  
    my_integers[i] = i;  
my_integers[7] = 9;           // undefined behaviour  
  
// reallocate array to larger size  
my_integers =  
    (int*)realloc(my_integers, sizeof(int) * 9);  
  
// write to new elements  
my_integers[7] = 100;         // now: OK  
my_integers[8] = 101;
```

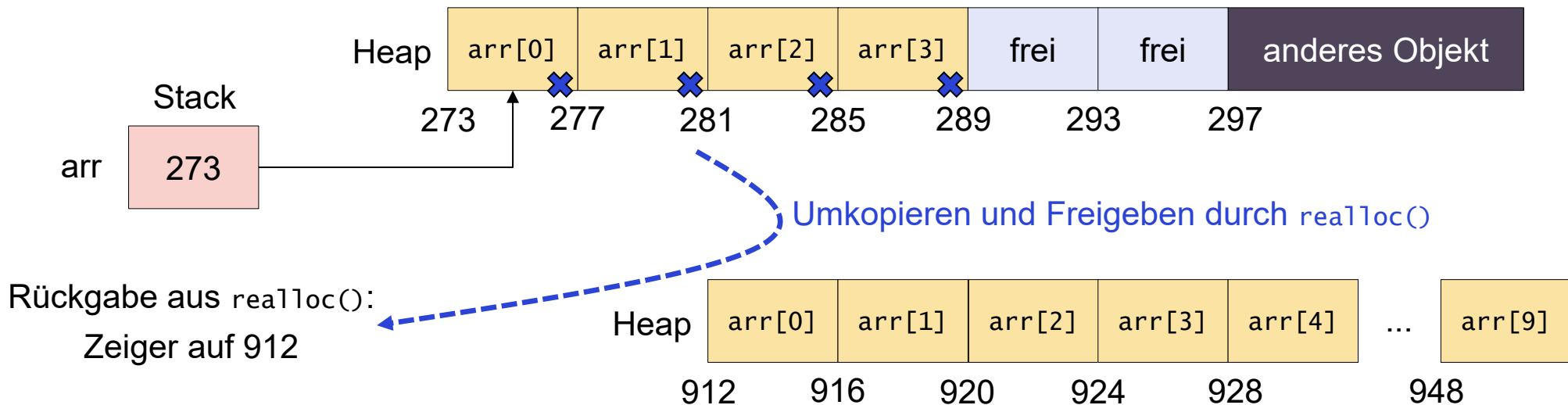
- mit `realloc()` lässt sich ein mit `malloc()` allozierter Zeiger nachträglich auf eine andere Größe umallokieren
- Größenangabe bei `realloc()` wie bei `malloc()`
- Ergebnisuweisung nach `realloc()` nicht vergessen!

Dynamische Arrays (4)

Verschiebung durch `realloc()`

```
float *arr = (float*)malloc(sizeof(float) * 4);  
realloc(arr, sizeof(float) * 10);  
arr[9] = 17.f; // undefined behaviour
```

zu wenig Platz
für 6 zusätzliche floats



- Arrayspeicher muss nach `realloc()` weiterhin zusammenhängen
- Verschiebung mittels Kopieren und Freigeben (eventuell, aber nicht immer) nötig
- Rückgabe aus `realloc()` ist (eventuell, aber nicht immer) anderer Zeiger
→ **Zuweisung nicht vergessen, sonst undefiniertes Verhalten!**

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- Arrays fester Größe
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Stolpersteine bei dynamischen Arrays (1)

Zeiger sind keine Arrays

```
int *array_a = (int*)malloc(sizeof(int) * 5);    // evtl. unsprechender Name
int *array_b = (int*)malloc(sizeof(int) * 5);    // evtl. unsprechender Name

for (int i = 0; i < 5; ++i)
    { array_a[i] = i; array_b[i] = i; }

if (array_a == array_b)                        // Zeigervergleich, kein elementweiser Vergleich!
    std::cout << "equal";                     // Ausgabe kommt nicht

array_a = array_b;                             // Zeigerzuweisung, keine tiefe Kopie!
array_b[0] = 10;
std::cout << array_a[0];                       // Ausgabe: 10

int *array_b_copy = (int*)malloc(sizeof(int) * 5); // unabhängige, tiefe Kopie
for (int i = 0; i < 5; ++i)
    array_b_copy[i] = array_b[i];
```

- ein (schlecht benannter) Zeiger bleibt ein Zeiger, auch wenn er auf ein Array zeigt
 - zugehörige Rechenoperationen (z. B. Zuweisung, Vergleich) arbeiten *flach* statt *tief*, d. h. es wird nur mit dem Zeiger gerechnet und nicht mit dem Array
 - tiefe Operationen (z. B. Kopie, Vergleich) sind bei Bedarf explizit zu programmieren (z. B. neuen Speicher allokalieren, einzelne Werte übertragen)

Stolpersteine bei dynamischen Arrays (2)

Fehlerfallen bei der Parameter-Semantik

```
// Allokiert drei Byte statt drei ints (also statt 12 Byte)!  
int *array = (int*)malloc(3);  
  
// sizeof(array) == sizeof(zeiger) == 4 (x86) oder 8 (x64)  
// Arraygrenze wird selbst nach Korrektur in malloc() überschritten!  
// → Schreiben in nicht allokierten Speicher  
// → undefiniertes Verhalten  
for (int i = 0; i < sizeof(array); ++i)  
    *(array + i) = (i * 17);  
  
// Lesen aus nicht allokiertem Speicher,  
// sofern man das malloc() nicht korrigiert!  
// → undefiniertes Verhalten  
std::cout << *(array + 3) << std::endl;  
  
free(array);
```

- Aufruf von `malloc()` **immer** mit `sizeof()`-Angabe im Parameter
- ein `sizeof(zeiger)` liefert Bytegröße von Zeiger, nicht von dynamischem Array
 - Fehlergefahr mit schlecht benannten Zeiger-Variablen
 - **Achtung:** vergleiche statisch dimensionierte Arrays (später im Vortrag)

Stolpersteine bei dynamischen Arrays (3)

Fehlende Initialisierung bei `malloc()`

```
int *iptr = (int*)malloc(sizeof(int));
std::cout << *iptr;           // standard: undefined behaviour
                               // usually: random value (e. g. -3, 11, or 1027)

int *array = (int*)malloc(sizeof(int) * 32);
std::cout << array[*iptr];     // undefined behaviour, use of random value as index

*iptr = 16;
std::cout << iptr[0];          // OK
std::cout << array[*iptr];     // OK
```

- der `malloc()`-Aufruf initialisiert Werte von allokierten Speicherzellen nicht
 - Vorabinitialisierung mit Standardwert (z. B. mit 0) ist oft unnötig und kostet Zeit
 - Praxis: bei Lesezugriff in aller Regel zufällige Werte
 - C++-Standard: bei Lesezugriff undefiniertes Verhalten

Faustregel:

Mit `malloc()` allokierten Speicher immer explizit initialisieren.

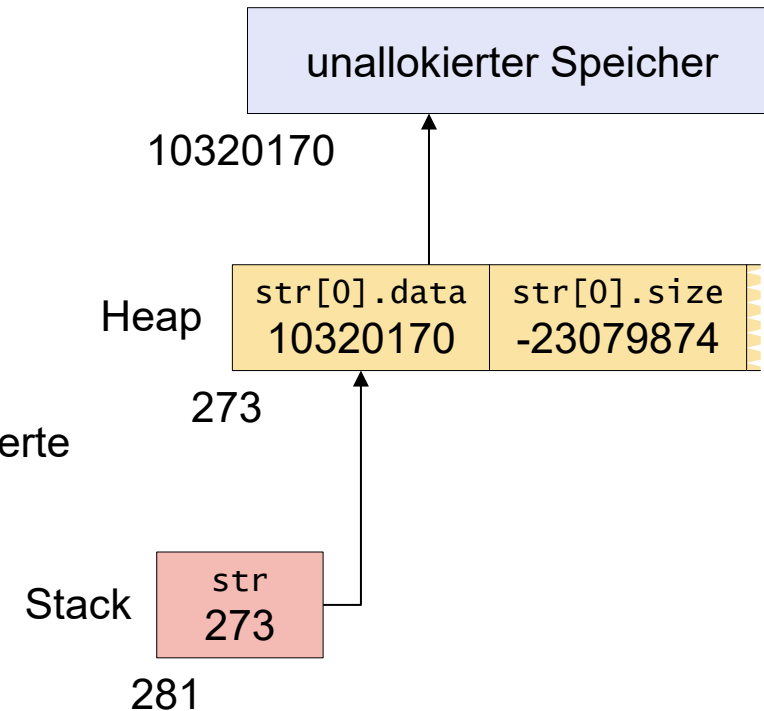
Stolpersteine bei dynamischen Arrays (4)

Fehlende Konstruktor-Aufrufe bei `malloc()`

```
std::string *str =           // allocate string array
    (std::string*)malloc(sizeof(std::string) * 6);

str[0] = "Hello";           // undefined behaviour
std::cout << *str;         // undefined behaviour
```

- Aufruf von `malloc()` ruft keine Konstruktoren auf
 - Membervariablen (z. B. Zeiger) haben zufällige Werte
 - häufig undefiniertes Verhalten beim Zugriff
- analog: keine Destruktoren bei `free()`
- analog: keine Kon- oder Destruktoren bei `realloc()`



Faustregel:

Für Klasseninstanzen kein `malloc()` verwenden.

- Empfehlungen: `std::vector<T>` oder `new T[array_size]`
- alternativ: `malloc()` und `placement-new`

Stolpersteine bei dynamischen Arrays (5)

Dynamische Allokationsarten in C und C++

```
int *i = new int;
int *j = new int[1];           // This is not the same as "new int"!
int *k = new int[10];
int *c = (int*)malloc(sizeof(int) * 10);

delete c;                      // undefined behaviour
delete[] c;                    // undefined behaviour
free(i); free(j); free(k);     // undefined behaviour
delete[] i;                    // undefined behaviour
delete j;                      // undefined behaviour

delete i; delete[] j; delete[] k; free(c);    // OK
```

Verwaltung von Heap-Allokationen z. B. mit:

- | | | | |
|-------------------|---------------------|---------------------------|--------------|
| • C | ptr = malloc(bytes) | ptr = realloc(ptr, bytes) | free(ptr) |
| • C++, ein Objekt | ptr = new T() | nicht möglich | delete ptr |
| • C++, Array | ptr = new T[count] | nicht möglich | delete[] ptr |

Heap-Allokationsarten nicht mischen!

→ sonst undefiniertes Verhalten

Stolpersteine bei dynamischen Arrays (6)

Zugriff nach Freigabe vermeiden

```
void print_data(int* array, int count)           // Hier fehlt Dokumentation:
{                                                 // diese Funktion gibt das Array frei!
    for (int i = 0; i < count; ++i)
        std::cout << array[i];

    free(array);

    if (!array)                                  // Kurzschreibweise:
        return;                                // if (ptr)      ⇔    if (ptr != nullptr)
                                                // if (!ptr)     ⇔    if (ptr == nullptr)

    free(array);                                // undefined behaviour
}

int *ptr = (int*)malloc(sizeof(int) * 10);
print_data(ptr, 10);
if (ptr)
    std::cout << ptr[0]; // undefined behaviour
```

- nach Freigabe mit `free()` ist weiterer Zugriff auf das Zeigerziel verboten
- Zeiger werden von `free()` nicht automatisch auf `nullptr` zurückgesetzt
→ vgl. `new` mit Zeigern für eine einzelne Zielvariable
(z. B. für Dokumentations-, Übergabe- und Rückgabekonventionen)

Stolpersteine bei dynamischen Arrays (7)

Sichere Umsetzung in `std::vector`

```
{
    int *p_a = (int*)malloc(sizeof(int) * 3);
    p_a[0] = 0;    p_a[1] = 1;    p_a[2] = 2;
    int *p_b = p_a;           // Zeigerzuweisung ("flache Kopie")
    p_b[1] = 3;
    std::cout << p_a[1];      // Änderung über p_b[1], Ausgabe: 3

    std::vector<int> v_a(3);
    v_a[0] = 0;    v_a[1] = 1;    v_a[2] = 2;
    std::vector<int> v_b = v_a; // Copy-Konstruktor (hier: "tiefe Kopie")
    v_b[1] = 3;
    std::cout << v_a[1];      // keine Änderung über v_b, Ausgabe: 1
} // memory leak for *p_a, *p_b
// no leak for v_a, v_b
```

Problem: C-Arrays (dynamische Arrays über Zeiger) sind (wie gesehen) fehleranfällig

Lösung: dynamisches Array in Klasse kapseln (z. B. als `std::vector<T>` direkt verfügbar)

→ automatisch tiefe Operationen (z. B. Vergleich, Kopie)

→ zusätzliche Sicherheit (z. B. Konstruktoraufrufe, korrekte Freigabe)

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- **Arrays fester Größe**
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Arrays fester Größe (1)

Arrays fester Größe auf dem Stack

```
struct point2f { float x, y; /* ... */ };

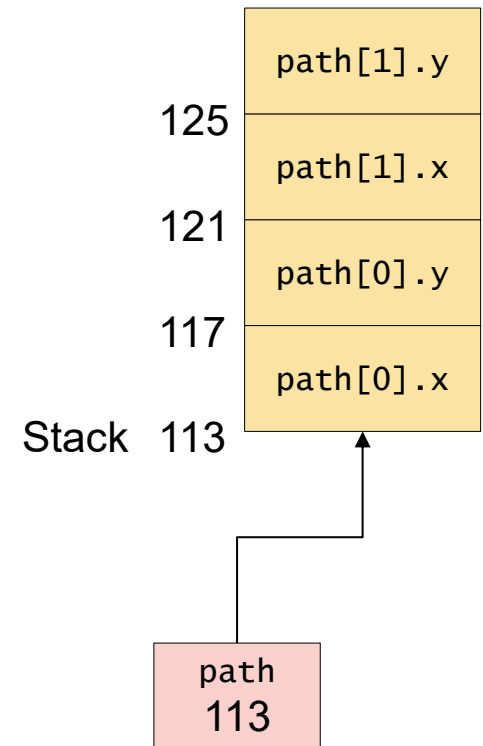
point2f path[2] =
    { point2f(-10, 17), point2f(4, 6) };

path[1].x = 21; // (*(path + 1)).x == (path + 1)->x

// attention: sizeof(array) != sizeof(ptr)
for (int i = 0; i < sizeof(path) / sizeof(path[0]); ++i)
    std::cout << path[i].x << " " << path[i].y;

point2f* path_ptr = path;
point2f* path_ptr_too = &path[0]; // path_ptr == path_ptr_too
```

- auf dem Stack können *Arrays fester Größe* angelegt werden
- Initialisierung über Zuweisung und geschweifte Klammern
- Array-Variable verhält sich nahezu wie Zeiger-Variable, aber:
 - `sizeof(array)` ist Gesamtgröße des Arrays in Byte
vgl.: `sizeof(ptr)` ist Größe des Zeigers in Byte
 - `array == &array == &array[0]` ist Anfangsadresse des Arrays
vgl.: `&ptr` ist Adresse der Zeiger-Variablen, `&ptr != &ptr[0]`



Achtung:
path-Zeiger
existiert nicht wirklich

Arrays fester Größe (2)

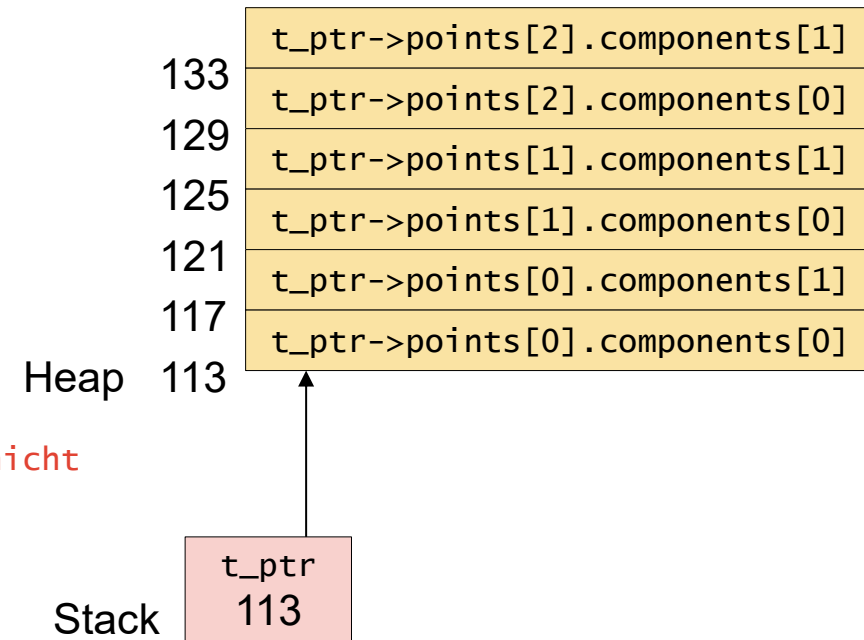
Arrays fester Größe als Member

```
struct point2f
{ float components[2]; /* ... */ };

struct triangle
{
    const int c_count = 3;
    point2f points[c_count];    // OK

    int count = 3;
    point2f points_err[count];  // kompiliert nicht
    /* ... */
};

triangle *t_ptr = new triangle();
```



- Arrays fester Größe können als Membervariablen auftreten
→ Verhalten wie Arrays fester Größe auf dem Stack
- Elemente des Arrays liegen wie einzelne Member direkt in umgebender Instanz
→ effizienter als separate (Heap-)Allokation (z. B. mit `malloc` oder `std::vector`)

Exkurs: Feste Array-Größe muss zur Compile-Zeit bekannt sein!

Arrays fester Größe (3)

Arrays fester Größe als Parameter und Rückgaben

```
void f_a(int *array) { /* ... */ } // evtl. Konvention: Zeiger auf nur einen int
void f_b(int array[]) { /* ... */ } // tatsächliche Signatur: f_b(int *array)
// oft Konvention: Zeiger auf ein oder mehr ints

void f_c(int array[5]) { /* ... */ } // tatsächliche Signatur: f_c(int *array)
// immer Konvention: Zeiger auf exakt 5 ints

void f_c(int *array) { /* ... */ } // compiler error: overload not allowed

int *ptr = (int*)malloc(sizeof(int) * 3);
f_a(ptr); // OK
f_b(ptr); // OK
f_c(ptr); // no compiler error, possibly bugged
```

- Arrays unbekannter und fester Größe als Funktionsparameter und -rückgaben möglich
- **Achtung Inkonsistenz:**

Hier wird kein vollständiges Array über den Stack übergeben...

... sondern die Schreibweise ist synonym zu einem gewöhnlichen Zeiger!

- keine Compilerprüfung
- Überladung nicht möglich
- Verwendung nur als zusätzliche Dokumentation

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- Arrays fester Größe
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Zeichenarrays und Strings (1)

ASCII-Zeichen

```
char c = 'x';  
std::cout << c;           // Ausgabe: x  
if (c == 120)             // OK, 120 ist Wert zu 'x' in ASCII-Tabelle  
    std::cout << "HERE";  // Ausgabe: "HERE" erscheint
```

Idee:

kodiere Buchstaben und andere Schriftzeichen als Ganzzahl-Werte

Umsetzung:

- *ASCII-Tabelle* mit Wertzuweisung für häufige, englische Zeichen als Konvention
- *ASCII-Zeichen* ist einzelnes Byte mit Datentyp `char`
- Schreibweise `'zeichen'` hat Typ `char` und ergibt Wert für `zeichen` aus ASCII-Tabelle
→ Lesbarkeit: bei Zeichen immer `'zeichen'` statt expliziten Wert verwenden!

Ausblick:

- für erweiterte Zeichen (z. B. Umlaute) gibt es mehrere, inkonsistente Zusatztabellen
- andere Zeichenkonventionen (z. B. UTF-8) lösen das Problem, sind aber in C++ hässlich

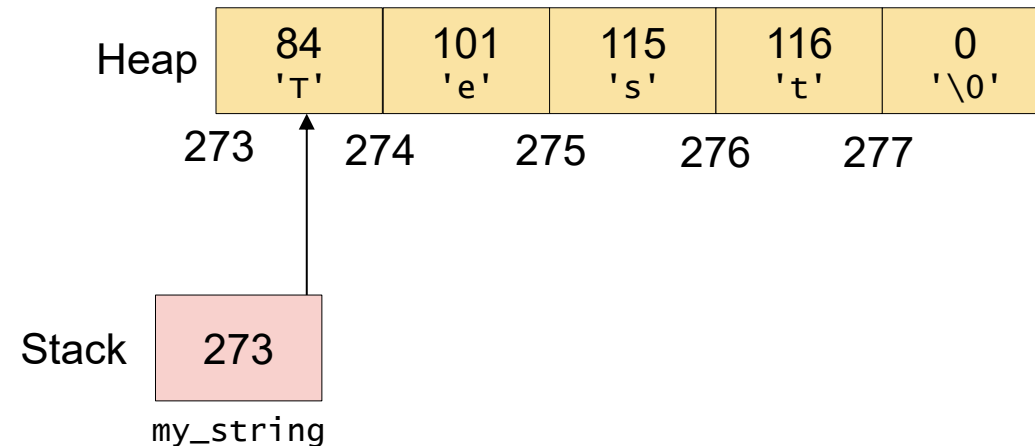
Zeichenarrays und Strings (2)

C-Strings als Array aus ASCII-Zeichen

```
// Zeichenarray allokieren
char* my_string =
    (char*)malloc(5 * sizeof(char));
```

```
// alle Zeichen initialisieren
my_string[0] = 'T';
my_string[1] = 'e';
my_string[2] = 's';
my_string[3] = 't';
my_string[4] = '\0';
```

```
// Übergabe als Zeiger,
// Länge wird von cout bei Bedarf
// über Suche nach \0 ermittelt
std::cout << my_string << std::endl;
```



- *C-String* (Zeichenarray) ist Array aus ASCII-Zeichen
 - Ablegen und Übergeben (z. B. an Funktionen) als Zeiger auf erstes Zeichen
 - Datentyp für Zeiger: `char*`
- letztes Zeichen ist immer `'\0'` ("null-terminated string")
 - vermeidet zusätzliche Variable (z. B. Parameter) für Länge

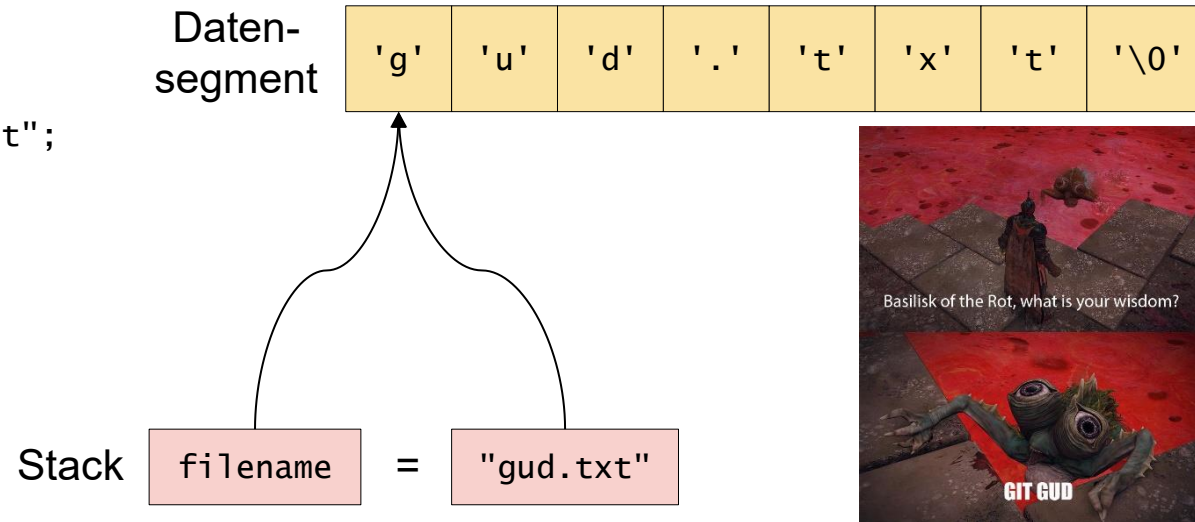
Zeichenarrays und Strings (3)

C-Strings und Zeichenliterale

```
// "gud.txt" ist Zeiger!  
const char * filename = "gud.txt";
```

```
std::ofstream output_file;  
output_file.open(filename);  
// Signatur: open(const char*)  
// Alternativ: open("gud.txt");
```

```
// undefiniertes Verhalten  
free(filename);
```



- **(C-)Zeichenliteral** `"text"` erzeugt anonymes, implizit `\0`-terminiertes ASCII-Zeichenarray
 - Ablage der Zeichen des Literals im Datensegment der `.exe`-Datei
 - Zeichenarray bleibt über gesamten Programmverlauf allokiert und gültig
 - keine manuelle Freigabe (z. B. mit `malloc` / `free`) zulässig oder nötig
- Programmausdruck `"text"` ergibt Zeiger auf das Zeichenarray
 - Datentyp ist zunächst `char*` (und nicht: `const char*`)
 - Verwendung von `"text"` überall dort möglich, wo ein `char*` stehen darf

Zeichenarrays und Strings (4)

Zeichenliterale und Konstantheit

```
char *message_a = "hello";           // Datentyp: char*
char* message_b = "hello";

message_a[0] = 'b';                  // undefiniertes Verhalten
std::cout << message_a << " " << message_b; // Ausgabe evtl.: bello bello
```

- Verwendung von `const char*` für Zeichenliterale wird nicht erzwungen
(Hintergrund: `const` kam erst im Sprachstandard C90 dazu)
- **Schreiben in Zeichen von Zeichenliteral führt zu undefiniertem Verhalten**
 - z. B. Zusammenfassen gleicher Literale → Änderung hat Seiteneffekte
 - z. B. Ablage der Literale in geschütztem Speicher → Änderung führt zu Absturz

Faustregel:

Zeiger auf Zeichenliterale immer als `const`-Zugriffsweg markieren!

Zeichenarrays und Strings (5)

C-Strings bei Zuweisung und Vergleich

```
char *string_a = "Test"; // evtl. unsprechender Name
char *string_b = (char*)malloc(sizeof(char) * 5); // evtl. unsprechender Name
string_b[0] = 'T'; string_b[1] = 'e'; string_b[2] = 's'; string_b[3] = 't';
string_b[4] = '\0';

if (string_a == string_b) // Zeigervergleich, kein zeichenweiser Vergleich!
    std::cout << "equal"; // Ausgabe kommt nicht

string_a = string_b; // Zeigerzuweisung, keine tiefe Kopie!
string_a[0] = 'R';
std::cout << string_b; // Ausgabe: "Rest"

char *string_b_copy = (char*)malloc(sizeof(char) * 5); // unabhängige, tiefe Kopie
for (int i = 0; i < 5; ++i)
    string_b_copy[i] = string_b[i];
```

- ein Zeiger auf eine Zeichenfolge ist ein Zeiger und wird als solcher verwendet
(z. B. flache Kopien, flache Vergleiche)
 - tiefe Operationen bei Bedarf selbst ausprogrammieren
 - STL-Hilfsfunktionen (z. B. strcmp, strlen, memcpy) für kompakteren Code nutzen

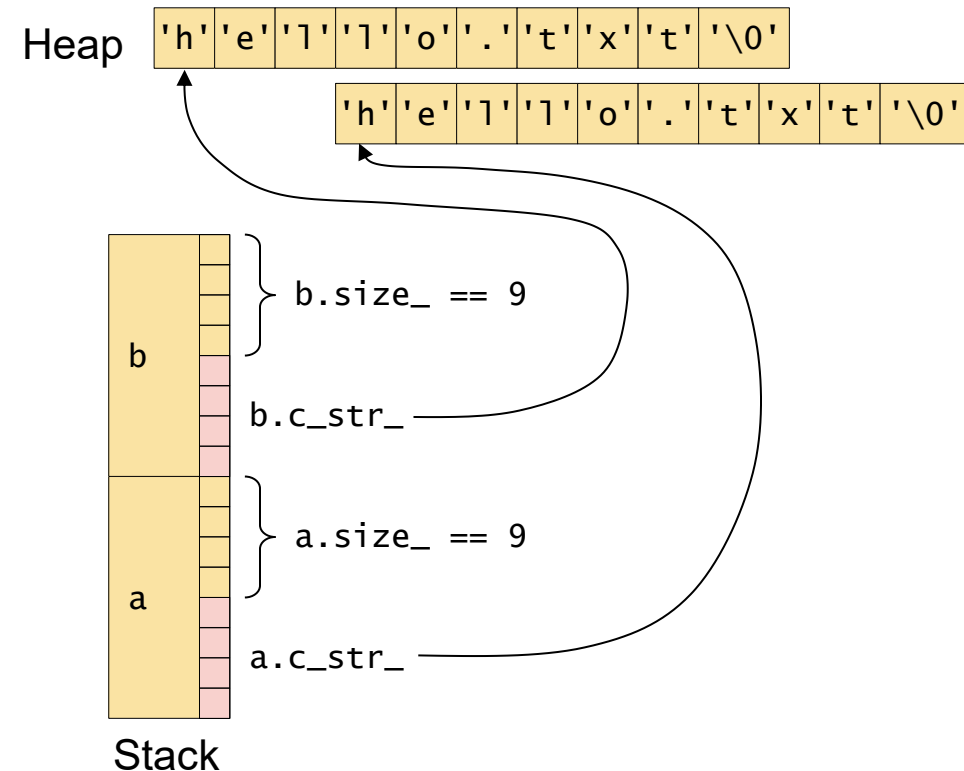
Zeichenarrays und Strings (6)

Kapselung in `std::string`

```
{
    std::string a("hello.txt"), b("");

    // kompakte std::string-Operationen,
    // Strings verhalten sich wie Werte
    b = a; b[0] = 'b';
    std::cout << a; // Ausgabe: "hello.txt"
    std::cout << b; // Ausgabe: "bello.txt"
    b[0] = 'h';
    if (a == b) std::cout << "Appears!";

    // getter für internen String,
    // z. B. für alte C-Funktionen
    FILE* f = fopen(a.c_str(), "rw");
} // automatische Heap-Speicherfreigabe
```



Vereinfachung:

- kapsle Verwaltung von String-Speicher in String-Klasse
- Klasse hält Zeiger für **kopierten** C-String (mit `\0`) und redundante Länge (ohne `\0`)
- featurearme Realisierung der Standardbibliothek (*C++-Strings*) in `std::string`
 - viele, alternative String-Klassen (z. B. `QString` aus QT, `cstringT` von Microsoft)

Vortragsübersicht

4 – Arrays

- Referenzen und Konstantheit
- Zeigerarithmetik
- Dynamische Arrays
- Stolpersteine bei dynamischen Arrays
- Arrays fester Größe
- Zeichenarrays und Strings
- Mehrdimensionale Arrays

Mehrdimensionale Arrays (1)

Nomenklatur

mehrdimensionales Array

- Arrayindex ist Tupel, z. B. $i \approx (x, y, z)$
- verschiedene Realisierungen möglich, z. B. mehrstufig oder linearisiert

mehrstufiges Array

- Ablage einzelner Zeilen und (Hyper-)Ebenen in separat allokierten Speicherblöcken
- jede Zeile und jede (Hyper-)Ebene kann eine individuelle Größe haben
- kompakte Zugriffssyntax
- höhere Allokationskosten (Rechenzeit, Speicherverbrauch, Codezeilen)

linearisiertes Array

- Ablage aller Elemente, Zeilen und (Hyper-)Ebenen in einem einzigen Speicherblock
- jede Zeile und jede (Hyper-)Ebene muss die gleiche Größe haben
- längliche (aber ähnlich schnelle) Zugriffssyntax
- geringere Allokationskosten (Rechenzeit, Speicherverbrauch, Codezeilen)

Mehrdimensionale Arrays (2)

Mehrstufige Arrays im Speicher

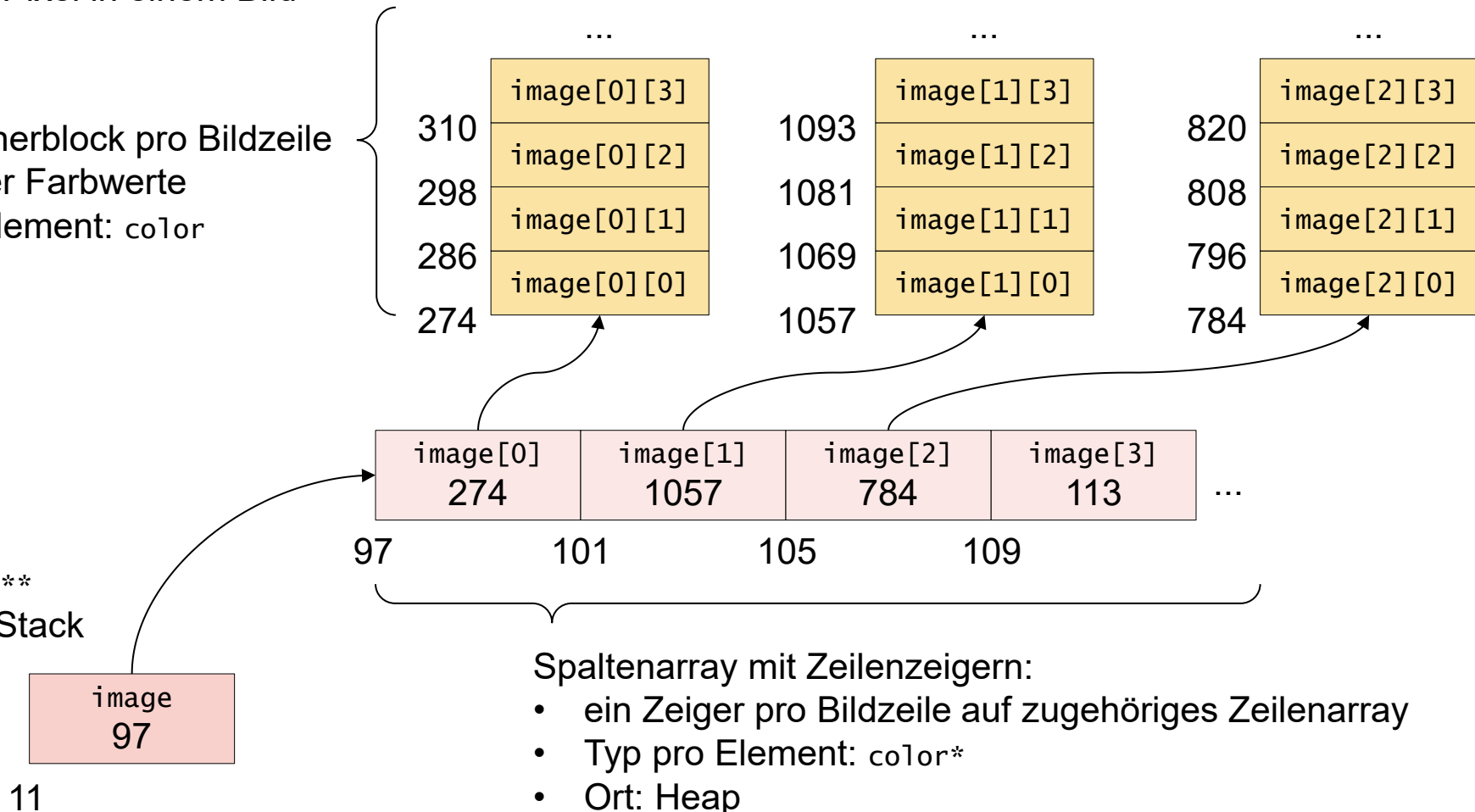
z. B.: Pixel in einem Bild

Zeilenarrays:

- ein Speicherblock pro Bildzeile
- Ablage der Farbwerte
- Typ pro Element: `color`
- Ort: Heap

Basiszeiger:

- Typ: `color**`
- Ort: z. B. Stack



Mehrdimensionale Arrays (3)

Mehrstufige Arrays im Code

```
struct color { unsigned char r, g, b; };
int size_x = 64, size_y = 64;

// Allokation
color **image =
    (color**)malloc(sizeof(color*) * size_y);

for (int y = 0; y < size_y; ++y)
    image[y] = (color*)malloc(sizeof(color) * size_x);

// Zugriff
image[py][px] = color(1, 37, 240);
std::cout << (*(image + py) + px).r << std::endl;

// Freigabe
for (int y = 0; y < size_y; ++y) free(image[y]);
free(image);
```

// image[py] ist Alias für Zeiger!

- mehrstufige Arrays bestehen aus (mehrfach) geschachtelten Zeigerarrays
- stufenweise Allokation und Freigabe
- Zugriff mit (mehrfacher) Zeigerarithmetik oder (mehrfacher) Arraysyntax

Mehrdimensionale Arrays (4)

Mehrstufige Arrays mit wechselnder Zeilenlänge

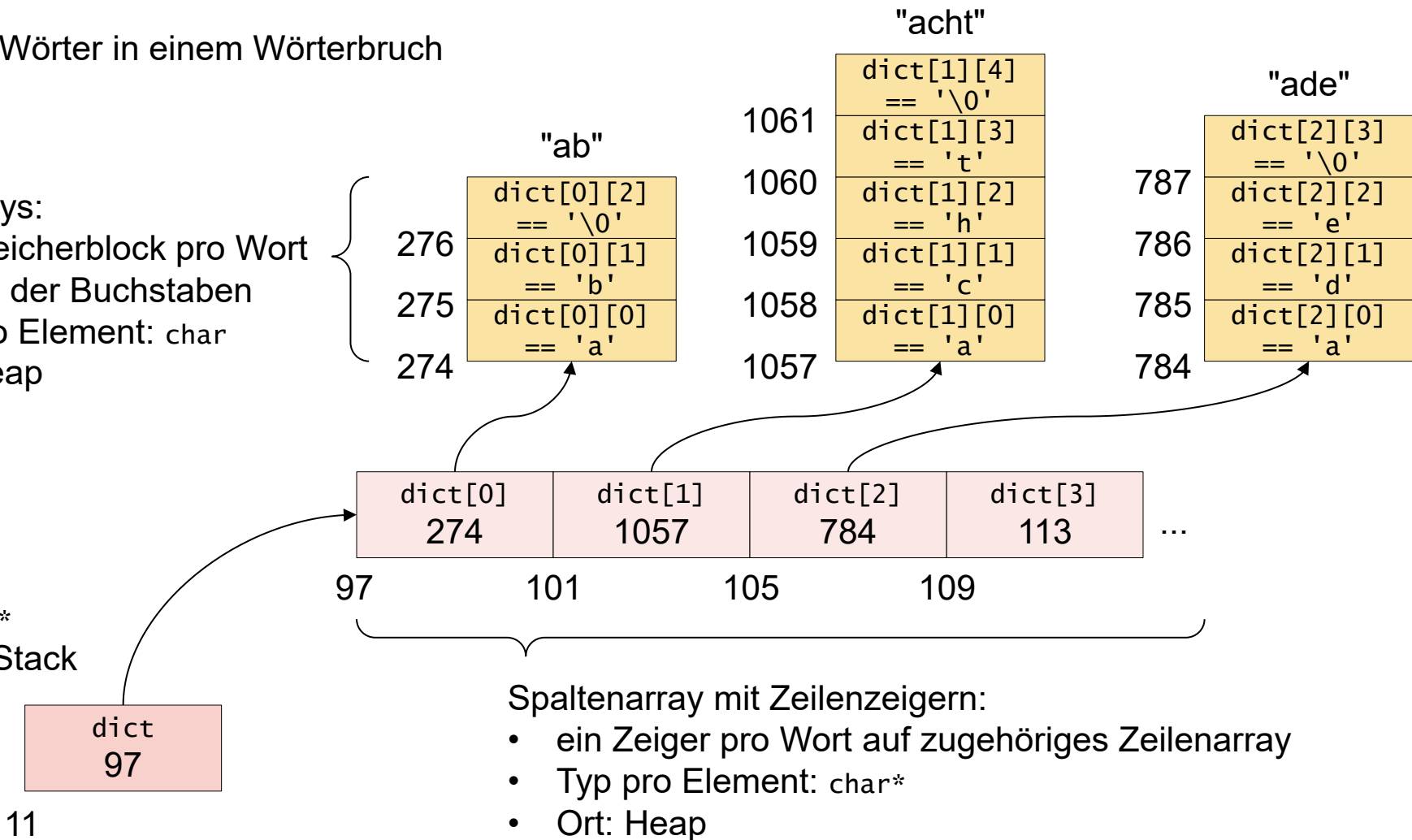
z. B.: Wörter in einem Wörterbruch

Zeilenarrays:

- ein Speicherblock pro Wort
- Ablage der Buchstaben
- Typ pro Element: char
- Ort: Heap

Basiszeiger:

- Typ: char**
- Ort: z. B. Stack



Spaltenarray mit Zeilenzeigern:

- ein Zeiger pro Wort auf zugehöriges Zeilenarray
- Typ pro Element: char*
- Ort: Heap

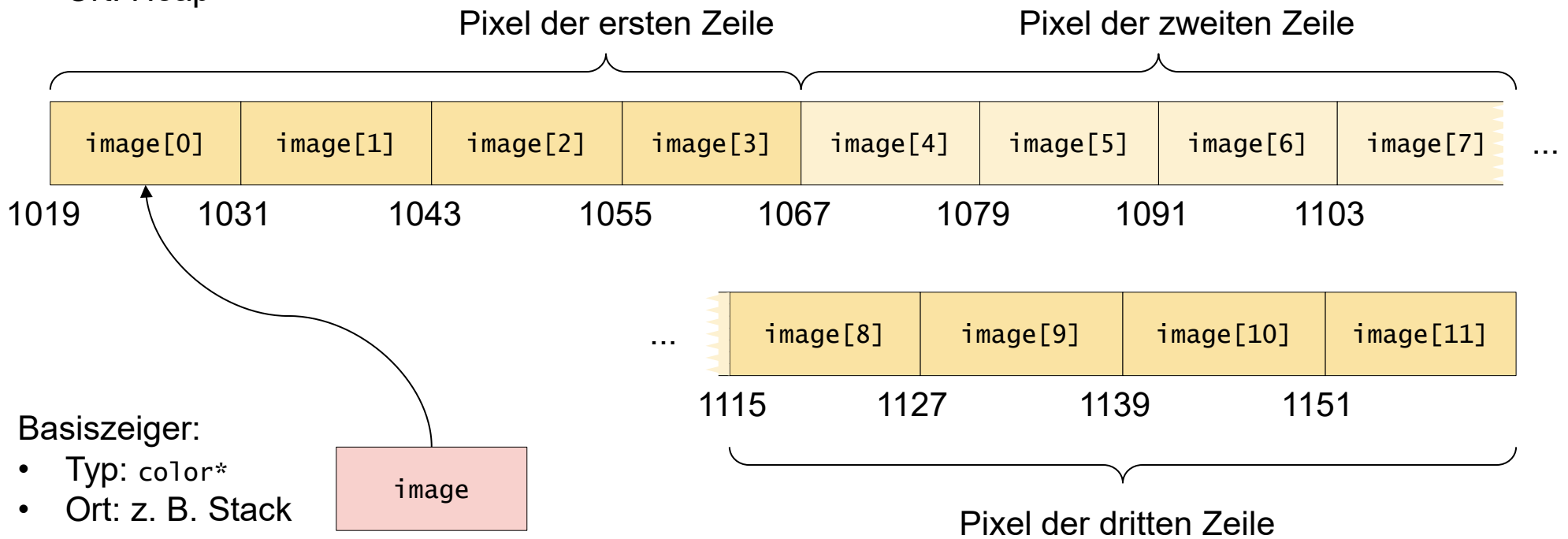
Mehrdimensionale Arrays (5)

Linearisiertes Array im Speicher

z. B.: Pixel in einem 3 Zeilen hohen und 4 Spalten breiten Bild

linearisiertes Array:

- Zeilen liegen hintereinander im selben Speicherblock
- Typ pro Element: `color`
- Ort: Heap



Mehrdimensionale Arrays (6)

Linearisiertes Array im Code

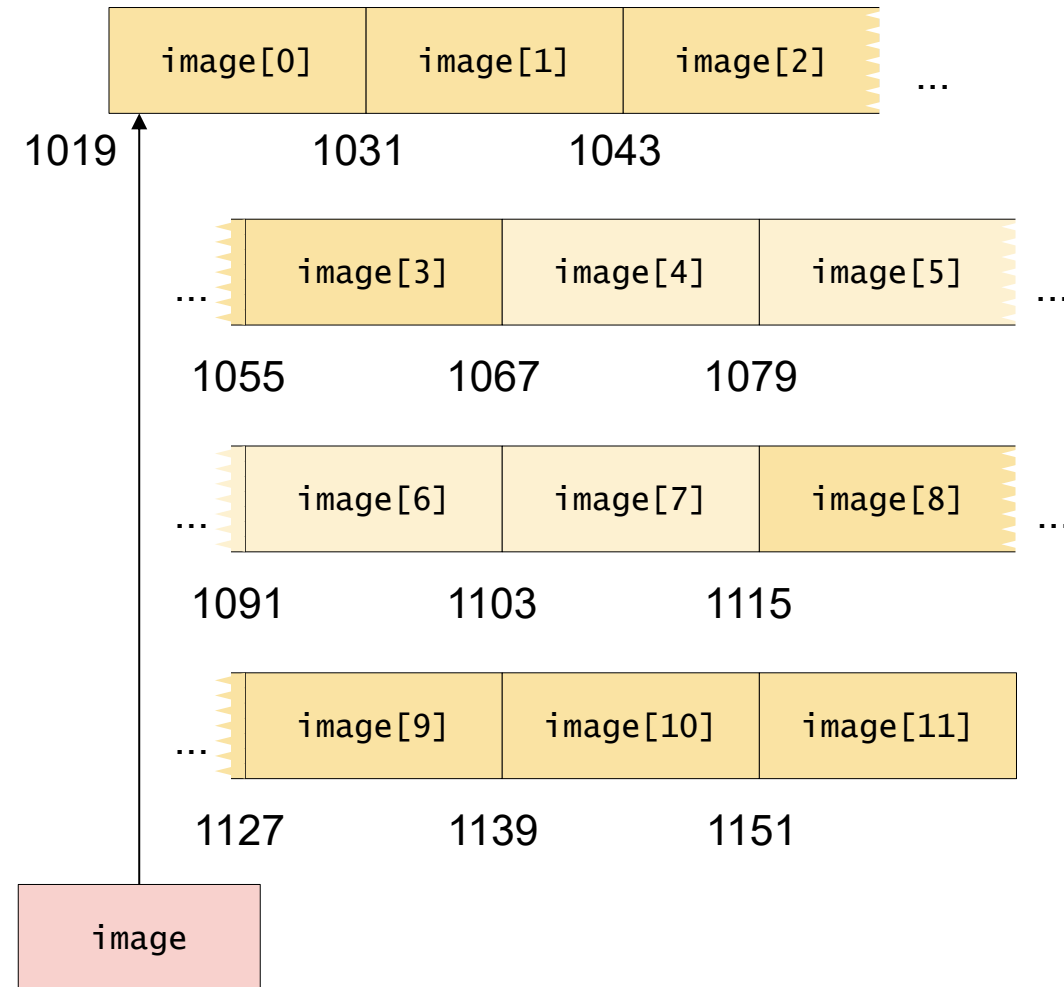
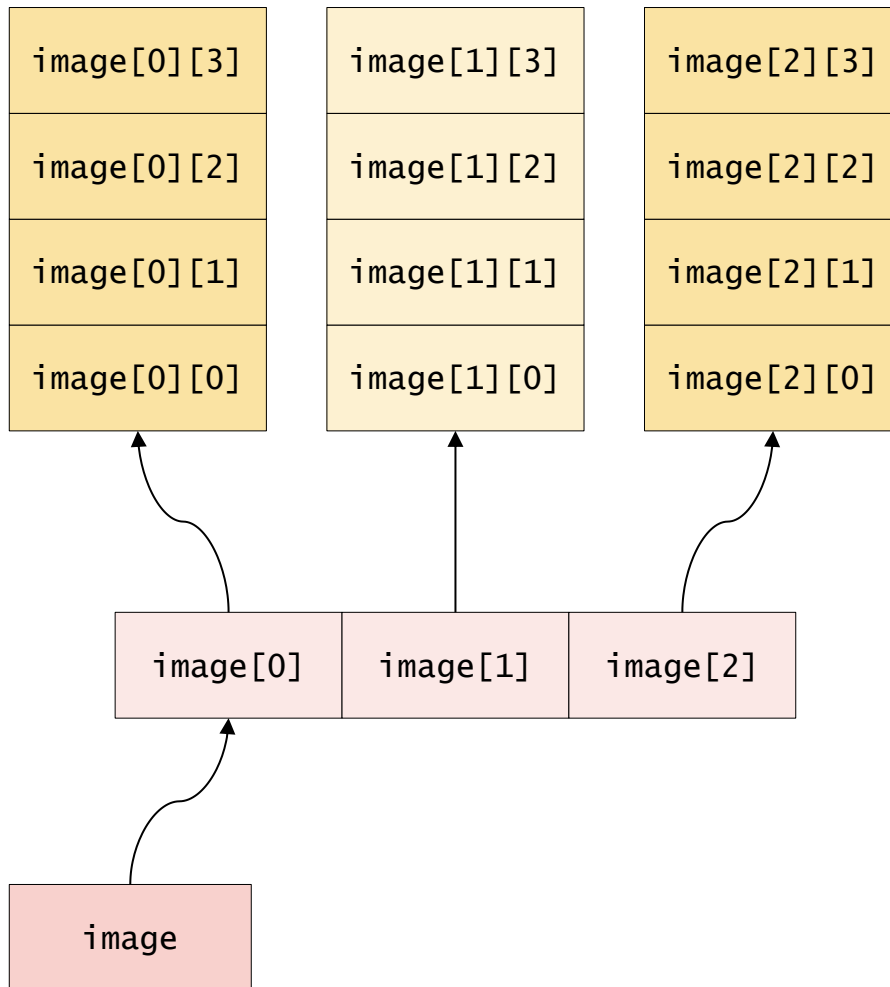
```
struct color { unsigned char r, g, b; };  
int size_x = 64, size_y = 64;  
  
// Allokation  
color *image =  
    (color*)malloc(sizeof(color) * size_x * size_y);  
  
// Zugriff  
image[py * size_x + px] = color(1, 37, 240);  
std::cout << *(image + py * size_x + px).r << std::endl;  
  
// Freigabe  
free(image);
```

- Allokation (und Freigabe) als ein Speicherblock
- Zugriff durch Bijektion auf eindimensionales (lineares) Array
 - Abbildung in eindimensionales Array $i = (x + y * \text{size_x})$
 - Rückabbildung mit Modularechnung $(x, y) = (i \% \text{size_x}, i / \text{size_x})$

→ beliebig viele, zusätzliche Dimensionen analog

Mehrdimensionale Arrays (7)

Gegenüberstellung mehrstufiges und linearisiertes Array



Zusammenfassung

Referenzen und Konstantheit

- Referenzen vergeben Alias für Ziel
- konstante Zugriffswege verhindern Änderung an Ziel

Zeigerarithmetik und Arrays

- Addieren von Offset zu Zeiger heißt Zeiger um Offset viele Elemente zu verschieben
- auf korrekte Arraygrößen und Zeigeroffsets achten
- Array durch Zeiger auf erstes Element und Zeigerarithmetik realisiert
- Zugriff mit `*(ptr + offset)` oder `ptr[offset]`
- auf Speicherverwaltung achten oder `std::vector` bzw. `std::string` verwenden

Verschiedenes

- Zeichenarrays sind `\0`-terminiertes Array aus Zeichen
- Arrays fester Größe ähneln Zeiger auf Stack, aber Achtung bei `sizeof()`
- mehrstufige Arrays durch Zeiger auf Zeiger realisiert
- linearisierte Arrays bilden mehrere Dimensionen auf lineares Array ab